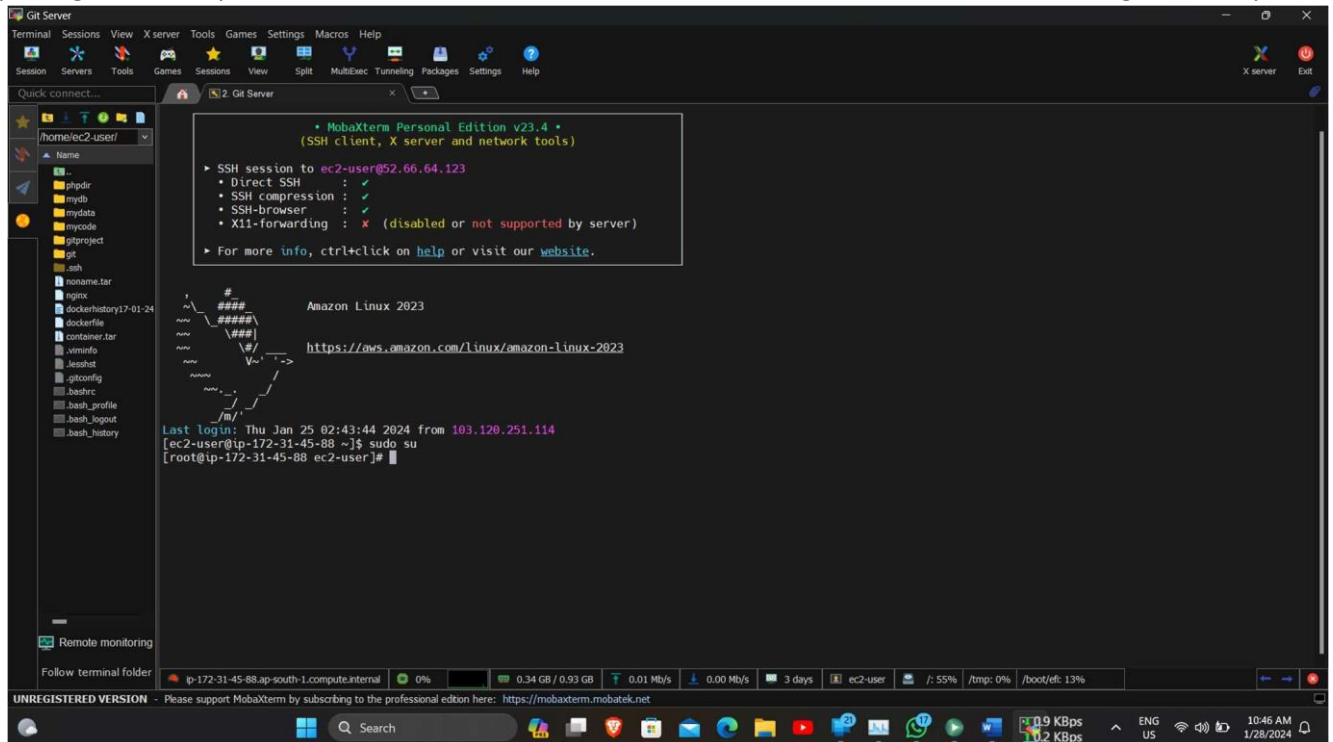


Q1. Pull Apache image from Docker hub & create own 'index.html' page.



Step1: Login as the superuser in the ec2-instance and make sure that the docker is installed and login correctly.



Step2: Use the following commands for the particular step

Step3: Pull 'nginx' docker image - `docker pull nginx`.

```
[root@ip-172-31-45-88 ec2-user]# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
2f44b7a888fa: Already exists
8b7dd3ed1dc3: Already exists
35497dd96569: Already exists
36664b6ce66b: Already exists
2d455521f76c: Already exists
dc9c4fdb83d6: Already exists
8056d2bcf3b6: Already exists
Digest: sha256:4c0fdaa8b6341bfdeca5f18f7837462c80c9ff90527ee35ef185571e1c327beac
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Step4: Create the docker container – `docker run -d --name mywebsite -p 80:80 nginx`

```
[root@ip-172-31-45-88 ec2-user]# docker run -d --name mywebsite -p 80:80 nginx
25f76c354757019214a4c1ca463f726dc66bdda7a203ed19ddb2ca3b4aa16eb0
```

Step5: Check the container is ready or not – `docker ps`

```
[root@ip-172-31-45-88 ec2-user]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS
25f76c354757   nginx     "/docker-entrypoint. ..." 17 seconds ago Up 16 seconds 0.0.0.0:80->80/tcp, :::80->80/tcp
```

Step6: Enter inside the docker container – `docker exec -it mywebsite /bin/bash`

Step7: change the working repository – `cd /usr/share/nginx/html/`

```
[root@ip-172-31-45-88 ec2-user]# docker exec -it mywebsite /bin/bash
root@25f76c354757:/# cd /usr/share/nginx/html/
root@25f76c354757:/usr/share/nginx/html# ls
50x.html  index.html
```

```
install nano -y
```

```
root@25f76c354757:/usr/share/nginx/html# apt update -y && apt install nano
Get:1 http://deb.debian.org/debian bookworm InRelease [151 kB]
Get:2 http://deb.debian.org/debian bookworm-updates InRelease [52.1 kB]
Get:3 http://deb.debian.org/debian-security bookworm-security InRelease [48.0 kB]
Get:4 http://deb.debian.org/debian bookworm/main amd64 Packages [8787 kB]
Get:5 http://deb.debian.org/debian bookworm-updates/main amd64 Packages [12.7 kB]
Get:6 http://deb.debian.org/debian-security bookworm-security/main amd64 Packages [134 kB]
Fetched 9185 kB in 2s (5887 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
```

index.html

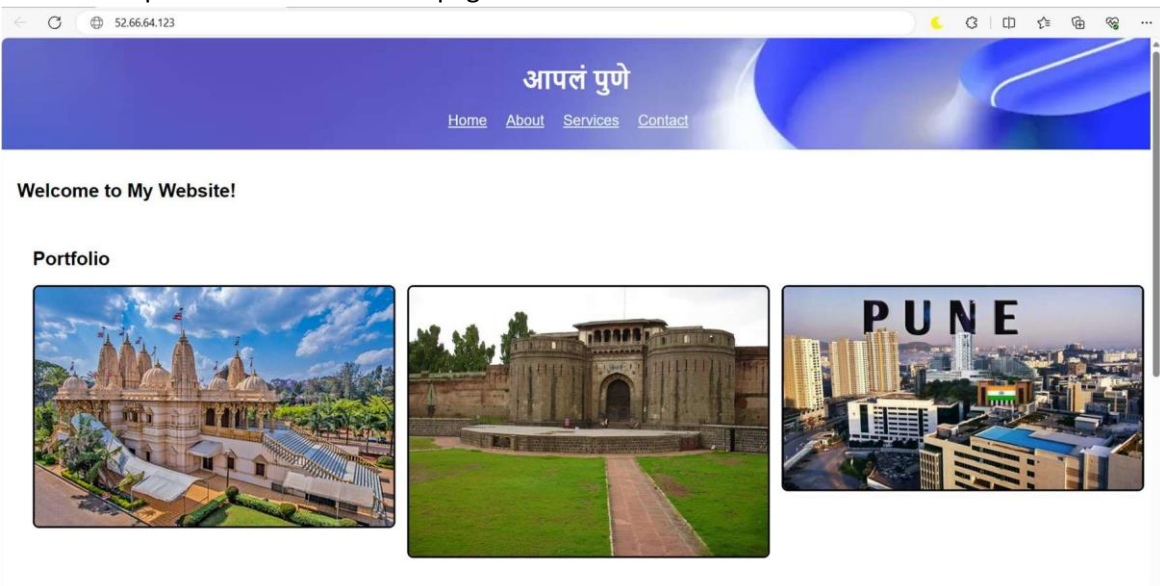
```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="styles.css">
    <title>Home Page</title>
</head>
<body>
    <header>
        <div>
            <a href="#">f</a>
            <a href="#">b</a>
            <a href="#">a</a>
            <a href="#">g</a>
        </div>
        <div>
            <a href="index.html">Home</a></div>
            <a href="about.html">About</a></div>
            <a href="services.html">Services</a></div>
            <a href="contact.html">Contact</a></div>
        </div>
    </nav>
</header>
<section class="main-content">
    <h2>Welcome to My Website!</h2>
    <section class="main-content">
        <h2>Portfolio</h2>
        <div class="portfolio-gallery">
            <div class="portfolio-item">
                
                <p>Swaminarayan Temple</p>
            </div>
            <div class="portfolio-item">
                
                <p>Shanivar Wada</p>
            </div>
            <div class="portfolio-item">
                
                <p>Pune</p>
            </div>
            <div class="portfolio-item">
                
                <p>Dagadushet ganapatip/p>
            </div>
            <!-- Add more portfolio items as needed -->
        </div>
    </section>
</section>
<footer>
    <p>©copy; 2023 Amol Tribhuwan. All rights reserved.</p>
</footer>
</body>
</html>

```

o10: save the file using **ctrl + s** key and exit using **ctrl + x**

011: hit the ip address and check the page is visible or not.



Q2. Create bind volume and mount for mysql, nginx, apache.



Step1: **Nginx -**

Step1: Create a directory to bind with container – **mkdir myvolume**

```
[root@ip-172-31-40-122 ec2-user]# mkdir myvolume
```

Step2: Check the directory are created or not—**ls**

```
[root@ip-172-31-40-122 ec2-user]# ls
myvolume
```

Step3: Pull the nginx docker image – **docker pull nginx**

```
[root@ip-172-31-40-122 myvolume]# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
2f44b7a888fa: Pull complete
8b7dd3ed1dc3: Pull complete
35497dd96569: Pull complete
36664b6ce66b: Pull complete
2d455521f76c: Pull complete
dc9c4fdb83d6: Pull complete
8056d2bcf3b6: Pull complete
Digest: sha256:4c0fdaa8b6341bfdeca5f18f7837462c80cff90527ee35ef185571e1c327beac
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Step4: Create the docker container –

docker run -d --name mynginx -p 80:80 -v /home/ec2-user/myvolume:/usr/share/nginx/html nginx

```
[root@ip-172-31-40-122 myvolume]# docker run -d --name mynginx -p 80:80 -v /home/ec2-user/myvolume:/usr/share/nginx/html nginx
4a5f7a03c188317d0d1059a573bc58bd7fef2d3206b1fbad17cbe31809a49689
```

Step5: Check the container is ready or not – **docker ps**

```
[root@ip-172-31-40-122 ec2-user]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
4a5f7a03c188   nginx    "/docker-entrypoint..." About a minute ago Up About a minute   0.0.0.0:80->80/tcp, :::80->80/tcp   mynginx
```

Step6: Change your workspace to directory you created and create index.html file –

cd myvolume && nano index.html

```
[root@ip-172-31-40-122 ec2-user]# cd myvolume/
[root@ip-172-31-40-122 myvolume]# nano index.html
```

Step7: Write your html test code to check whether is applicable or not & after writing code press **ctrl + s** and **ctrl + x** to save the file and exit from nano text editor.

```
GNU nano 5.8 index.html
<h1>page created to demonstarait the bind volume</h1>
```

Step8: Hit the ip address of the server from any web browser to test the page.

page created to demonstarait the bind volume

- **mysql –**

Step1: Create a new directory mysqlvolume – **mkdir mysqlvolume**

```
[root@ip-172-31-40-122 ec2-user]# mkdir mysqlvolume
```

Step2: Check the directory are created or not – **ls**

```
[root@ip-172-31-40-122 ec2-user]# ls
mysqlvolume  myvolume
```

Step3: Pull the mysql docker image – **docker pull mysql**

```
[root@ip-172-31-40-122 ec2-user]# docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
558b7d69a2e5: Pull complete
2cb5a921059e: Pull complete
b85878fb9bb2: Pull complete
d16f3fd26a82: Pull complete
afd51b5329cb: Pull complete
374d2f7f3267: Pull complete
4ea1bb2c9574: Pull complete
1c9054053605: Pull complete
d79cd2da03be: Pull complete
e3a1aa788d17: Pull complete
Digest: sha256:d7c20c5ba268c558f4fac62977f8c7125bde0630ff8946b08dde44135ef40df3
Status: Downloaded newer image for mysql:latest
docker.io/library/mysql:latest
```

Step4: Create the mysql container – **docker run -d --name mysql -v /home/ec2-user/mysqlvolume:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123 mysql**

```
[root@ip-172-31-40-122 ec2-user]# docker run -d --name mysql -v /home/ec2-user/mysqlvolume:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123 mysql
a25878c20ffe7c17e014c96b636539d1f96271eb341456f0fb01ab7aa7d03b6c
```

Step5: Check the state if container is running or not – **docker ps**

```
[root@ip-172-31-40-122 ec2-user]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
a25878c20ffe   mysql     "docker-entrypoint.s..." 54 minutes ago Up 54 minutes 3306/tcp, 33060/tcp               mysql
```

Step6: go to inside the container – **docker exec -it mysql /bin/bash**

```
bash-4.4# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.3.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Step7: create 1 database in it – **create database facebook;**

```
mysql> create database facebook;
Query OK, 1 row affected (0.01 sec)
```

Step8: view the database is created or not – **show databases;**

```
mysql> show databases;
+-----+
| Database |
+-----+
| facebook |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.01 sec)
```


Step9: exit from mysql and check for the database files in the mysql container –

```
exit
cd /var/lib/mysql
ls
```

```
mysql> exit
Bye
bash-4.4# cd /var/lib/mysql
bash-4.4# ls
'#ib_16384_0.dblwr'  '#innodb temp'  binlog.000002  ca.pem          facebook        ibtmp1         mysql.sock      public_key.pem  sys
'#ib_16384_1.dblwr'  auto.cnf        binlog.index   client-cert.pem ib_buffer_pool  mysql          performance_schema  server-cert.pem  undo_001
'#innodb_redo'       binlog.000001  ca-key.pem     client-key.pem  ibdata1        mysql.ibd      private_key.pem  server-key.pem   undo_002
bash-4.4#
```

Step10: exit from the container and check the files in the mysqlvolume name directory –

```
exit
ls
cd mysqlvolume
ls
```

```
bash-4.4# exit
exit
[root@ip-172-31-40-122 ec2-user]# ls
mysqlvolume myvolume
[root@ip-172-31-40-122 ec2-user]# cd mysqlvolume/
[root@ip-172-31-40-122 mysqlvolume]# ls
'#ib_16384_0.dblwr'  '#innodb temp'  binlog.000002  ca.pem          facebook        ibtmp1         mysql.sock
'#ib_16384_1.dblwr'  auto.cnf        binlog.index   client-cert.pem ib_buffer_pool  mysql          performance_schema
'#innodb_redo'       binlog.000001  ca-key.pem     client-key.pem  ibdata1        mysql.ibd      private_key.pem
[root@ip-172-31-40-122 mysqlvolume]#
```

- Apache –

Step1: Create the myapache directory – `mkdir myapache`

```
[root@ip-172-31-40-122 ec2-user]# mkdir myapache
```

Step2: Pull the apache docker image on the server – `docker pull httpd`

```
[root@ip-172-31-40-122 ec2-user]# docker pull httpd
Using default tag: latest
latest: Pulling from library/httpd
2f44b7a888fa: Already exists
376771e8483c: Pull complete
4f4fb700ef54: Pull complete
6a6627aefcf0: Pull complete
152f4888b550: Pull complete
fd0579f22872: Pull complete
Digest: sha256:ba846154ade27292d216cce2d21f1c7e589f3b66a4a643bff0cdd348efd17aa3
Status: Downloaded newer image for httpd:latest
docker.io/library/httpd:latest
```

Step3: If your previously running container mapped to the 80 port you stop remove all the containers that mapped to it –

```
docker stop $(docker ps -aq) && docker rm $(docker ps -aq)
```

```
[root@ip-172-31-40-122 ec2-user]# docker stop $(docker ps -aq) && docker rm $(docker ps -aq)
a25878c20ffe
4a5f7a03c188
a25878c20ffe
4a5f7a03c188
[root@ip-172-31-40-122 ec2-user]#
```

Step4: Check the directory is created or not and create the docker container –

```
ls
docker run -d --name myapache -p 80:80 -v /home/ec2-user/myapache:/usr/local/apache2/htdocs httpd
```

```
[root@ip-172-31-40-122 ec2-user]# ls
myapache mysqlvolume myvolume
[root@ip-172-31-40-122 ec2-user]# docker run -d --name myapache -p 80:80 -v /home/ec2-user/myapache:/usr/local/apache2/htdocs httpd
0cddd77d9b8fb698b430492f6fc753e6a114f851e5e2b03d33b87c0d3270deae
[root@ip-172-31-40-122 ec2-user]#
```

Step5: Change your workspace to the created directory – `cd myapache`

```
[root@ip-172-31-40-122 ec2-user]# cd myapache
```

Step6: Create index.html file to test the directory is bind or not – `nano index.html`

```
GNU nano 5.8 index.html Modified
^I>This example is for the apache docker container bind volume written in the myapache diirectory index.html file</h1>
```

Step7: Hit the ip address from any web-browser to check the volume is bind or not

A screenshot of a web browser window. The address bar shows a 'Not secure' warning icon followed by the IP address '35.154.35.53'. The browser interface includes back, forward, and search buttons, as well as star, print, and menu icons.

This example is for the apache docker container bind volume written in the myapache diirectory index.html file

Q3. Create named volume and mount for mysql, nginx, apache.



- Nginx –

Step1: Create docker volume – `docker volume create mynginx`

```
[root@ip-172-31-40-122 myapache]# docker volume create mynginx
mynginx
```

Step2: Check the volume is created or not – `docker volume ls`

```
[root@ip-172-31-40-122 myapache]# docker volume ls
DRIVER      VOLUME NAME
local       mynginx
```

Step3: Pull the nginx docker image – `docker pull nginx`

```
[root@ip-172-31-40-122 myapache]# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
2f44b7a888fa: Pull complete
8b7dd3ed1dc3: Pull complete
35497dd96569: Pull complete
36664b6ce66b: Pull complete
2d455521f76c: Pull complete
dc9c4fdb83d6: Pull complete
8056d2bcf3b6: Pull complete
Digest: sha256:4c0fdaa8b6341bfdeca5f18f7837462c80cff90527ee35ef185571e1c327beac
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Step4: Create docker container –

`docker run -d --name mynginx -v mynginx:/usr/share/nginx/html -p 80:80 nginx`

```
[root@ip-172-31-40-122 myapache]# docker run -d --name mynginx -v mynginx:/usr/share/nginx/html -p 80:80 nginx
21b381c45dc5dd3d78e8cce3d83d68c7d6f9338c3f959036c0f0d9f7d0ce992a
```

Step5: Go inside the container—`docker exec -it mynginx /bin/bash`

```
[root@ip-172-31-40-122 ec2-user]# docker exec -it mynginx /bin/bash
```

Step6: Change your workspace to the html working directory –

`cd /usr/share/nginx/html`

`ls`

```
root@21b381c45dc5:/# cd /usr/share/nginx/html
root@21b381c45dc5:/usr/share/nginx/html# ls
50x.html index.html
```

Step7: Create a new file in workspace & check it created or not –

`touch mynewfile.html`

`ls`

```
root@21b381c45dc5:/usr/share/nginx/html# touch mynewfile.html
root@21b381c45dc5:/usr/share/nginx/html# ls
50x.html index.html mynewfile.html
```

Step8: Exit from the container and go to check the file are tracked or not –

`exit`

`cd /var/lib/docker/volumes/mynginx/_data`

`ls`

```
root@21b381c45dc5:/usr/share/nginx/html# exit
exit
[root@ip-172-31-40-122 ~]# cd /var/lib/docker/volumes/mynginx/_data
[root@ip-172-31-40-122 _data]# ls
50x.html index.html mynewfile.html
```

- Mysql –

Step1: Create docker volume – `docker volume create mysql`

```
[root@ip-172-31-40-122 _data]# docker volume create mysql
mysql
```

Step2: Check is the docker volume is created or not – `docker volume ls`

```
[root@ip-172-31-40-122 _data]# docker volume ls
DRIVER      VOLUME NAME
local       mynginx
local       mysql
```

Step3: Pull the mysql docker image – `docker pull mysql`

```
[root@ip-172-31-40-122 _data]# docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
558b7d69a2e5: Pull complete
2cb5a921059e: Pull complete
b85878fb9bb2: Pull complete
d16f3fd26a82: Pull complete
afd51b5329cb: Pull complete
374d2f7f3267: Pull complete
4ea1bb2c9574: Pull complete
1c9054053605: Pull complete
d79cd2da03be: Pull complete
e3a1aa788d17: Pull complete
Digest: sha256:d7c20c5ba268c558f4fac62977f8c7125bde0630ff8946b08dde44135ef40df3
Status: Downloaded newer image for mysql:latest
docker.io/library/mysql:latest
```

Step4: Create mysql docker container –

`docker run -d --name mysql -v mysql:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123 mysql`

```
[root@ip-172-31-40-122 _data]# docker run -d --name mysql -v mysql:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123 mysql
b623a607130ab49fdb5b7308b3eff6657354ba056243bb00f39fbb2962764f10
```

Step5: go inside of the container – `docker exec -it mysql /bin/bash`

```
[root@ip-172-31-40-122 _data]# docker exec -it mysql /bin/bash
```

Step6: login to the database – `mysql -u root -p`

```
bash-4.4# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.3.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Step7: create one database and check it –

`create database facebook;`

`show databases;`

```
mysql> create database facebook;
Query OK, 1 row affected (0.01 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| facebook |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.01 sec)
```


Step8: exit from the database and container also –

```
exit
exit
```

```
mysql> exit
Bye
bash-4.4# exit
exit
```

Step9: check for the files is tracked by docker volume or not –

```
cd /var/lib/docker/volumes/mysql/_data/
```

```
[root@ip-172-31-40-122 _data]# cd /var/lib/docker/volumes/mysql/_data/
```

Step10: check the file – ls

```
[root@ip-172-31-40-122 _data]# ls
'#ib_16384_0.dblwr'  '#innodb_temp'  binlog.000002  ca.pem          facebook        ibtmp1         mysql.sock      public_key.pem  sys
'#ib_16384_1.dblwr'  auto.cnf        binlog.index   client-cert.pem ib_buffer_pool  mysql          performance_schema  server-cert.pem  undo_001
'#innodb_redo'       binlog.000001  ca-key.pem     client-key.pem  ibdata1        mysql.ibd      private_key.pem  server-key.pem   undo_002
[root@ip-172-31-40-122 _data]#
```

- Apache –

Step1: Create docker volume – `docker volume create myapache`

```
[root@ip-172-31-40-122 _data]# docker volume create myapache
myapache
```

Step2: Check if the docker volume is created or not – `docker volume ls`

```
[root@ip-172-31-40-122 _data]# docker volume ls
DRIVER      VOLUME NAME
local       myapache
local       mynginx
local       mysql
```

Step3: Pull apache docker image – `docker pull httpd`

```
[root@ip-172-31-40-122 _data]# docker pull httpd
Using default tag: latest
latest: Pulling from library/httpd
2f44b7a888fa: Already exists
376771e8483c: Pull complete
4f4fb700ef54: Pull complete
6a6627aecff0: Pull complete
152f4888b550: Pull complete
fd0579f22872: Pull complete
Digest: sha256:ba846154ade27292d216cce2d21f1c7e589f3b66a4a643bff0cdd348efd17aa3
Status: Downloaded newer image for httpd:latest
docker.io/library/httpd:latest
```

Step4: Create mysql docker container –

```
docker run -d --name myapache -v myapache:/usr/local/apache2/htdocs httpd
```

```
[root@ip-172-31-40-122 _data]# docker run -d --name myapache -v myapache:/usr/local/apache2/htdocs httpd
fcef4ca85050bcc32d0ad5e2c24417e0f3ac33a7a7d56f9db8de59fa90e37a7d
```

Step5: go inside of the container – `docker exec -it myapache /bin/bash`

```
[root@ip-172-31-40-122 _data]# docker exec -it myapache /bin/bash
```

Step6: change your workspace to the html working directory –

```
cd /usr/local/apache2/htdocs
```

```
root@fcef4ca85050:/usr/local/apache2# cd /usr/local/apache2/htdocs
```

Step7: update the package manager and install the text editor –

`apt update && apt install nano -y`

```
root@fcef4ca85050:/usr/local/apache2/htdocs# apt update && apt install nano -y
Get:1 http://deb.debian.org/debian bookworm InRelease [151 kB]
Get:2 http://deb.debian.org/debian bookworm-updates InRelease [52.1 kB]
Get:3 http://deb.debian.org/debian-security bookworm-security InRelease [48.0 kB]
Get:4 http://deb.debian.org/debian bookworm/main amd64 Packages [8787 kB]
Get:5 http://deb.debian.org/debian bookworm-updates/main amd64 Packages [12.7 kB]
Get:6 http://deb.debian.org/debian-security bookworm-security/main amd64 Packages [134 kB]
Fetched 9185 kB in 2s (5905 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Step8: create the new file — `nano new.html`

```
root@fcef4ca85050:/usr/local/apache2/htdocs# nano new.html
```

Step9: Write your html test code to check whether is applicable or not & after writing code press `ctrl + s` and `ctrl + x` to save the file and exit from nano text editor.

```
GNU nano 5.8 new.html
a:cozco
```

Step10: exit from the container – `exit`

```
root@fcef4ca85050:/usr/local/apache2/htdocs# exit
exit
```

Step11: change the directory – `cd /var/lib/docker/volumes/myapache/_data`

```
[root@ip-172-31-40-122 _data]# cd /var/lib/docker/volumes/myapache/_data
```

Step12: list the files – `ls`

```
[root@ip-172-31-40-122 _data]# ls
index.html  new.html
```